

LAB 13.3 Transformer – Hugging Face – BERT

2403A52006

MD.Mustafa

✓ Install Required Libraries

```
!pip install transformers datasets -q

import torch
import numpy as np
import pandas as pd

from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)

from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

✓ Load Dataset (IMDb)

```
"""
STEP 3: Load IMDb Dataset
Binary Sentiment Classification (Positive / Negative)
"""

dataset = load_dataset("imdb")

print(dataset)
print("\nSample Record:\n")
print(dataset["train"][0])
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
README.md:      7.81k/? [00:00<00:00, 295kB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable token authentication.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable token authentication.

plain_text/train-00000-of-00001.parquet: 21.0M/21.0M [00:00<00:00, 21.2MB/s] 100%

plain_text/test-00000-of-00001.parquet: 20.5M/20.5M [00:00<00:00, 48.2MB/s] 100%

plain_text/unsupervised-00000-of-00001.p(...): 42.0M/42.0M [00:00<00:00, 49.9MB/s] 100%

Generating train split: 100% 25000/25000 [00:00<00:00, 86543.56 examples/s]

Generating test split: 100% 25000/25000 [00:00<00:00, 101539.92 examples/s]

Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 110930.97 examples/s]

DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})

Sample Record:

{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that sur

```

✓ Check Class Distribution

```

"""
STEP 4: Check Class Distribution
"""

train_labels = dataset["train"]["label"]

unique, counts = np.unique(train_labels, return_counts=True)

print("Class Distribution:")
for u, c in zip(unique, counts):
    print(f"Label {u}: {c}")

Class Distribution:
Label 0: 12500
Label 1: 12500

```

✓ Load BERT Tokenizer

```

"""
STEP 5: Load Pretrained BERT Tokenizer
"""

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

```

```

config.json: 100%                               570/570 [00:00<00:00, 58.3kB/s]
tokenizer_config.json: 100%                       48.0/48.0 [00:00<00:00, 5.32kB/s]
vocab.txt: 100%                                   232k/232k [00:00<00:00, 2.40MB/s]
tokenizer.json: 100%                              466k/466k [00:00<00:00, 4.20MB/s]

```

✓ Tokenization Function

```

"""
STEP 6: Tokenize Text
Apply padding and truncation
"""

def tokenize_function(example):
    return tokenizer(
        example["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

tokenized_dataset = dataset.map(tokenize_function, batched=True)

```

```

Map: 100%                               25000/25000 [00:39<00:00, 604.93 examples/s]
Map: 100%                               25000/25000 [00:36<00:00, 1093.83 examples/s]
Map: 100%                               50000/50000 [00:45<00:00, 1212.94 examples/s]

```

✓ Prepare Dataset for PyTorch

```

"""
STEP 7: Convert Dataset to PyTorch Format
"""

tokenized_dataset = tokenized_dataset.remove_columns(["text"])
tokenized_dataset = tokenized_dataset.rename_column("label", "labels")
tokenized_dataset.set_format("torch")

train_dataset = tokenized_dataset["train"]
test_dataset = tokenized_dataset["test"]

```

✓ Load Pretrained BERT Model

```

"""
STEP 8: Load BERT Model for Classification
"""

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)

```

```

model.safetensors: 100% 440M/440M [00:03<00:00, 280MB/s]

Loading weights: 100% 199/199 [00:00<00:00, 605.09it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expected
- MISSING :those params were newly initialized because missing from the checkpoint. Consider tra

```

✓ Define Evaluation Metrics

```

"""
STEP 9: Define Evaluation Metrics
Accuracy, Precision, Recall, F1
"""

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average="binary"
    )

    acc = accuracy_score(labels, predictions)

    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }

```

✓ Define Training Arguments

```

training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=100
)

`logging_dir` is deprecated and will be removed in v5.2. Please set `TensorBoardLoggingDir` inst

```

Initialize Trainer

```
"""
STEP 11: Initialize Trainer
"""

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
```

Train Model

```
"""
STEP 12: Train the Model
"""
```

```
trainer.train()
```

 [2067/3126 17:29 < 08:58, 1.97 it/s, Epoch 1.32/2]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.300377	0.283295	0.882840	0.904557	0.856000	0.879609

Writing model shards: 100% 1/1 [00:06<00:00, 6.18s/it]

Writing model shards: 100% 1/1 [00:01<00:00, 1.42s/it]

Writing model shards: 100% 1/1 [00:06<00:00, 6.57s/it]

Writing model shards: 100% 1/1 [00:01<00:00, 1.53s/it]

 [3126/3126 29:04, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.300377	0.283295	0.882840	0.904557	0.856000	0.879609
2	0.172328	0.340464	0.892800	0.885158	0.902720	0.893853

Writing model shards: 100% 1/1 [00:05<00:00, 5.73s/it]

Writing model shards: 100% 1/1 [00:14<00:00, 14.09s/it]

Writing model shards: 100% 1/1 [00:06<00:00, 6.62s/it]

```
TrainOutput(global_step=3126, training_loss=0.24884199013102917, metrics={'train_runtime':
1744.7873, 'train_samples_per_second': 28.657, 'train_steps_per_second': 1.792, 'total_flos':
3328881072414720.0, 'train_loss': 0.24884199013102917, 'epoch': 2.0})
```

Evaluate Model

```
"""
STEP 13: Evaluate Model Performance
"""
```

```
results = trainer.evaluate()
print("\nEvaluation Results:\n", results)
```

[1563/1563 03:26]

Evaluation Results:

Generate Confusion Matrix

```
{ 'eval_loss': 0.3404642641544342, 'eval_accuracy': 0.8928, 'eval_precision': 0.885158456228428, 'eval_recall': 0.8928 }
```

```
"""
STEP 14: Confusion Matrix Heatmap
"""

predictions = trainer.predict(test_dataset)
pred_labels = np.argmax(predictions.predictions, axis=1)
true_labels = predictions.label_ids

cm = confusion_matrix(true_labels, pred_labels)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

